

T2 - Tutorial: DES and AES encryption using OpenSSL

Goals of this tutorial

- 1) To show how symmetric encryption ciphers can be used by means of the **OpenSSL** software and then to investigate some usage details.
- 2) To check the **performance** of modern CPUs when running the DES and AES ciphers.
- 3) To investigate the **assumption** (reported both on the book and the slides) that 10^6 decryptions/ μ s are realistic for brute-forcing the encryption key.

Using the DES cipher

In GNU/Linux, encrypting a text (or a generic file) using the DES cipher is very easy. In fact, we can simply use the implementation provided by the OpenSSL software tool (warning: DES is an obsolete and insecure cipher, **never use it in your applications!**).

First of all, we create a new file named "*plaintext.txt*" that contains some text.

`echo "Hello world!" > plaintext.txt`

Now, we can proceed with the encryption operation¹.

`openssl enc -e -des -salt -in plaintext.txt -out ciphertext.bin -pass pass:cybersecurity`

We obtain a new binary file "*ciphertext.bin*" that contains the encrypted text and a **warning** about a problem with the "key derivation". **What is the key derivation?**

Since the obtained encrypted output is a binary file then it is not possible to visualize its content by using a text editor but it is possible to visualize its content using the "xxd" command. As said above, the cipher used for encrypting the message is the outdated DES algorithm. This is a relevant security issue but there is **at least another** relevant security aspect that must be

openssl enc: Invoca lo strumento di cifratura di OpenSSL.
-e: Specifica l'operazione di encryption. Per decifrare si userebbe -d.
-des: Seleziona l'algoritmo DES. (Nota: è obsoleto e insicuro).
-salt: Aggiunge un valore casuale (salt) durante la derivazione della chiave.
-in plaintext.txt: Il file di input da proteggere.
-out ciphertext.bin: Il file di output binario risultante.
-pass pass:cybersecurity: Fornisce la password direttamente nel comando (in questo caso "cybersecurity").

Avoid using simple dictionary words for your password.

Respect to plaintext.txt size, it increases a lot the size because there is a little bit of metadata

Weak Key Derivation (KDF): The "Warning" you received is a major OpSec red flag. Older OpenSSL versions use a single iteration of a hash function to turn your password into a key.

giving us an error

¹ Recent versions of OpenSSL have disabled the standard use of DES. This cipher can be enabled again creating a local text file named "openssl.cnf" with the content provided by [this page](#) (section "Loading Providers"). After that, and before running the OpenSSL command reported above, it is necessary to execute the following command: `> export OPENSSL_CONF="openssl.cnf"` (to instruct OpenSSL to use the newly provided configuration file). Enabling the Legacy Provider Configuration allows the use of deprecated ciphers (such as DES). This should be restricted strictly to forensic analysis, legacy data recovery or educational purposes. The use of obsolete algorithms in production environments poses a serious security risk, as their cryptographic strength is negligible in the face of modern computing power.

a Key Derivation Function (KDF) is the bridge between a human-readable password (like "cybersecurity") and a fixed-length binary key required by a cipher (like the 56 bits needed for DES). Algorithms like DES or AES cannot use "words" as keys; they require a specific number of raw bits. The KDF is the mathematical process that transforms your password into those bits.

While using an obsolete cipher like DES is the most obvious flaw, the operational security (OpSec) failure is much deeper.

1) Default Mode of Operation (The ECB Problem): When you use `openssl enc -des` without specifying a mode, OpenSSL often defaults to ECB. As we discussed, ECB is deterministic. It encrypts each block of data independently without any "memory" of the previous block. This results in pattern leakage. If your `plaintext.txt` contains repeated data, those patterns will be perfectly visible in the ciphertext. An attacker doesn't need to decrypt the file to understand its structure or guess its contents.

2) Password Exposure in Command History: The command uses `-pass pass:cybersecurity`. On Linux systems, the commands you type are stored in a plain-text file. Any user with access to your machine (or any script with read permissions) can simply type history and see your password in clear text. Furthermore, the password appears in the system's process list (`ps -aux`) while the command is running. To mitigate this problem, use a file or type it in the standard input (not in the command line)

considered. In other words, the operation that we have performed above is insecure in terms of operational security. **What is wrong with it?**

Now, we can proceed with the decryption operation.

```
openssl enc -d -des -in ciphertext.bin -out plaintext2.txt -pass pass:cybersecurity
```

If everything worked as expected, the decrypted text is now in "`plaintext2.txt`" and we can verify if this file is byte-to-byte identical with the plaintext file.

```
diff plaintext2.txt plaintext.txt
```

The ciphers provided by OpenSSL are many and it is possible to list them with:

```
openssl help
```

Not only standard ciphers are available

Moreover, we can use the benchmarks that are integrated into OpenSSL to check the encryption / decryption speed of our CPU. This can be very helpful to evaluate which cipher is better suited for our hardware and furthermore we can investigate some baseline assumptions used in the textbook. For example, in some parts of the book, it is assumed that the hardware of a potential attacker is able to perform $\sim 10^6$ **decryptions/ μ s**. Is this decryption speed realistic or not?

We discovered that a single CPU is not able to perform the operations at the level of speed that we discussed before

```
> openssl speed
```

With this command, we are measuring the execution speed of OpenSSL on the computer we are using, for all available encryption algorithms, over a given period of time (the default is 3 seconds, unless otherwise specified)

```
> openssl speed <cipher>
```

Limit the above command on a specific cipher (with all of its configurations: key size, block size, etc...). For a period of around 3 seconds per block size, OpenSSL "feeds" data into the processor, encrypting it as quickly as possible. It does not write anything to disk: everything takes place in RAM and on the CPU to measure pure computing power.

The output for each cipher configuration is the number of bytes it can encrypt within the specified time

I suggest being very careful in assessing the output that is provided by the benchmarks. Another aspect that is fundamental is the option that permits to execute in parallel more instances of the benchmark. In other words, it is possible to execute in parallel (on each CPU core that is available) more decryption instances of the software.

```
> openssl speed -multi n <cipher>
```

We can parallelise the exploration of the universe of the keys, because each key is independent from the others. This means that if you want to test all the keys: You do not need to follow a specific order. No communication is required between the different processors (no dependency). You can divide the universe into 'slices' and assign one to each core or machine.

REMEMBER: in this case, we are not discussing about the Mode of operation, but about brute-forcing a key

In the previous command, the "**n**" parameter represents the number of **instances to be executed in parallel**. It should be obvious that running more instances than the available CPU cores does not lead to a speedup but a slowdown. Executing the encryption in parallel is very important for reducing the amount of time needed for encrypting a file or, for example, to get higher throughput in (encrypted) communications. In fact, in some systems, the encryption / decryption procedure could be a performance bottleneck. Regarding an attacker that is willing to break an encrypted message, it is obvious that speeding up the key search by means of running more instances of the decryption in parallel is a practical option.

Il confronto con l'attaccante teorico: Il libro assume che un attaccante possa fare 10^6 decifrazioni al microsecondo. Se prendi il tuo dato (797117 byte/s con des-cbc con 64 block size) e lo dividi con 10^6 , che indica il passaggio a microsecondi: Il tuo risultato indica quante decifrazioni il TUO computer fa al microsecondo. Se il tuo numero è, ad esempio, 0,8, significa che una single CPU è più lento dell'attaccante ipotizzato dal libro. Realismo dell'assunzione: Per una single CPU standard: No, non è realistico l'assunzione. Una singola CPU moderna, come hai visto dai tuoi test, è ordini di grandezza più lenta. Per un attaccante professionale: Sì, è realistico. Un attaccante non usa una CPU generica, ma utilizza: GPU (Schede video): Possono fare migliaia di operazioni in parallelo. ASIC (Circuiti dedicati): Chip costruiti solo per decifrare il DES. Questi hardware possono facilmente raggiungere e superare la soglia di un milione di decifrazioni al microsecondo.

Using the AES cipher

We will compare the throughput of DES against AES (AES is incredibly faster than DES because every core of the CPU has an hardware implementation of AES)

In the book, it is reported that the software implementations of DES are sluggish in terms of performance. ^{Quindi} Hence it is interesting to compare the execution speed of **DES** with what can be obtained by a more modern cipher like **AES**.

→ `openssl speed aes-128-cbc`

Many modern processors provide a hardware implementation of AES. **This is now used by default in OpenSSL** (i.e., the CPU implementation of the cipher is considered as trusted). The effect is a very relevant increase in the performance that can be obtained and it is worth noting that (typically) every CPU core has an independent AES hardware unit that can be used in parallel.

If there is a software implementation, open source, of the cipher, you can check it. If there is an hardware implementation of the cipher, you are trusting the implementation without the possibility to check it (you give only input and key and the cipher is implemented in hardware): the only thing that you can do is checking the output of the hardware implementation with a software one and check if it is the same. If you don't trust the hardware or software implementation of the cipher used in OpenSSL, you can change it (but only with software ones, if you have only an hardware implementation: so you will lose performance)